Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа №4

За шестой семестр

По дисциплине: «Модели решения задач в интеллектуальных системах»

Тема: «Классификация линейно разделимых образов однослойным персептроном для логических функций n переменных»

Выполнил:
Студент 3 курса
Группы ИИ-26(1)
Прокопюк А.Д.
Проверила:
Андренко К.В.

Брест 2026

Цель работы: Реализовать нейронную сеть (однослойный персептрон) для решения задач синтеза логических функций от n входов. Сравнить эффективность обучения при различных начальных условиях и параметрах шага.

Задачи лабораторной работы:

1. Сгенерировать полную таблицу истинности для заданной логической функции (2n наборов). На выходе функции должны использоваться значения 0 и 1.
2. **Реализовать алгоритм обучения:**
   - Разделить выборку на обучающую (часть данных) и тестовую (оставшиеся наборы) для проверки обобщающей способности.
   - В качестве функции активации использовать сигмоиду
   - Провести **два эксперимента** по обучению:
     А. С фиксированным шагом обучения
     Б. С адаптивным шагом
   - Выход сети интерпретировать как вероятность принадлежности к классу «1».
   - Обучение проводить до минимизации суммарной ошибки на обучающей выборке.
3. **Провести анализ сходимости:**
   - Построить графики зависимости суммарной ошибки от номера эпохи для обучающей и тестовой выборок.
   - Оценить обобщающую способность: корректно ли сеть предсказывает значения на тех наборах из таблицы истинности, которые она не видела при обучении.
   - Зафиксировать количество эпох, необходимых для настройки весов при разных значениях
4. Вывести итоговые значения весовых коэффициентов и порога.
5. **Реализовать режим функционирования:**
   - Программа должна принимать на вход вектор из n чисел (0 или 1).
   - Сеть должна мгновенно выдавать результат (вероятность и округленный класс).
6. Сделать вывод о способности сигмоидального персептрона представлять базовые логические операции и восстанавливать полную таблицу истинности по частичной выборке.

№ of variant	n	Task
12	7	AND^i

Код программы:

```
import numpy as np

import matplotlib.pyplot as plt


class Perceptron:
    def __init__(self, input_size=0, learning_rate=0.1, target_accuracy=1e-6,
X_test=None, Y_test=None):
        self.X = np.array([])
```

```
        self.w = np.random.uniform(-1.0, 1.0, input_size + 1)
        self.learning_rate = learning_rate
        self.target = np.array([])
        self.target_accuracy = target_accuracy
        self.X_test = X_test
        self.Y_test = Y_test

    def set_X(self, X: np.array) -> None:
        X = np.array(X)

        if X.ndim == 1:
            X = X.reshape(1, -1)
        elif X.ndim != 2:
            print("X must be 1D or 2D array")
            return

        self.X = X
        self.X = np.insert(self.X, 0, -1, axis=1)

    def set_target(self, target: np.array) -> None:
        if self.X.ndim != 2:
            print("X not setted!")
            return

        if len(target) != len(self.X):
            print(f"Invalid size of target vector. It must have length =
{len(self.X)}. Now = {len(target)}")
            return

        self.target = target

    def get_wsum(self, X: np.array) -> np.array:
        return np.dot(X, self.w)

    def activate(self, arr_wsum: np.array) -> np.array:
        act = 1 / (1 + np.exp(-arr_wsum))
        return np.where(act >= 0.5, 1, 0)

    def prediction(self, X_input=None) -> np.array:
        X = self.X if X_input is None else X_input

        if X.size == 0:
            print("Input X vector not set!")
            return None

        wsum = self.get_wsum(X)
        y = 1 / (1 + np.exp(-wsum))

        return y

    def delta(self, y: np.array) -> None:
```

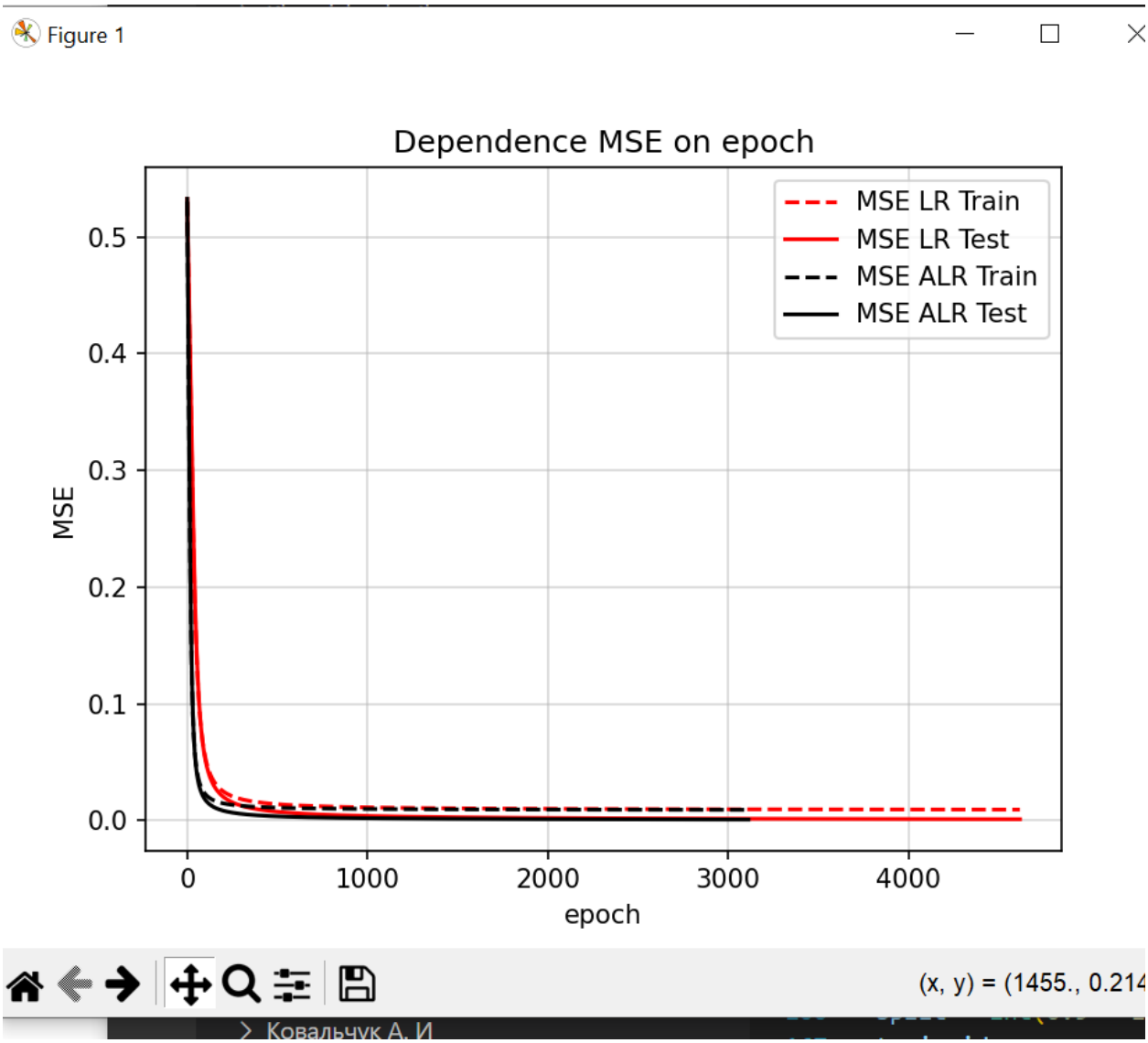
```
        error = y - self.target
        der = y * (1-y)
        self.w = self.w - self.learning_rate * np.dot(error * der, self.X) /
len(error)

    def mse(self, y: np.array, test_mode=False) -> float:
        if test_mode:
            return np.mean((y - self.Y_test) ** 2)
        return np.mean((y - self.target) ** 2)

    def evaluate(self) -> np.array:
        X_test = np.insert(self.X_test, 0, -1, axis=1)
        y = self.prediction(X_test)
        return self.mse(y, True)

    def train(self, epochs=500, patience=10) -> np.array:   # поезд
        mse_history = []
        mse_test_history = []

        for epoch in range(epochs):
            y = self.prediction(self.X)

            mse = self.mse(y)
            mse_history.append(mse)

            if self.X_test is not None:
                mse_test_history.append(self.evaluate())

            if mse <= self.target_accuracy:
                print(f"Final for [LR{self.learning_rate}]:\nEpoch:
{epoch}\nMSE:{mse:.8f}")
                break

            if len(mse_history) > patience:
                recent = mse_history[-10:]
                if max(recent) - min(recent) < self.target_accuracy:
                    print(f"[LR{self.learning_rate}] Stoped on {epoch}
epoch.\n[REASON] No progress on {patience} epochs.")
                    print(f"[LR] threshold: {self.w[0]}")
                    print(f"[LR] weigths: {self.w[1:]}")
                    break

            self.delta(y)

        return mse_history, mse_test_history

    def train_adptive_lr(self, epochs=500, patience=10) -> np.array:   # поезд
        mse_history = []
        mse_test_history = []
        self.learning_rate = 1 / np.mean(np.sum(self.X**2, axis=1))
```

```
        for epoch in range(epochs):
            y = self.prediction(self.X)

            mse = self.mse(y)
            mse_history.append(mse)

            if self.X_test is not None:
                mse_test_history.append(self.evaluate())

            if mse <= self.target_accuracy:
                print(f"Final for [ALR] ({self.learning_rate}):\nEpoch:
{epoch}\nMSE:{mse:.8f}")
                break

            if len(mse_history) > patience:
                recent = mse_history[-10:]
                if max(recent) - min(recent) < self.target_accuracy:
                    print(f"[ALR{self.learning_rate}] Stoped on {epoch}
epoch.\n[REASON] No progress on {patience} epochs.")
                    print(f"[ALR] treshold: {self.w[0]}")
                    print(f"[ALR] weigths: {self.w[1:]}")
                    break

            self.delta(y)

        return mse_history, mse_test_history

    def run(self, input_vector: list) -> None:
        x = np.array(input_vector).reshape(1, -1)
        x = np.insert(x, 0, -1, axis=1)
        prob = self.prediction(x)[0]
        cls = 1 if prob > 0.5 else 0
        print(f"Probability: {prob:.4f} | Class: {cls}")

N = 7
X = np.arange(2**N). reshape(-1,1)
X = (X >> np.arange(N-1, -1, -1)) & 1

indices = np.random.permutation(2**N)
X_shuffle = X[indices]

train_size = int(len(X) * 0.9)

Y = np.zeros(2**N)
inverse_arr = np.array([1, 0, 0, 0, 0, 0, 0]) # mask
for i in range(len(Y)):
    row = X[i].copy()
    for j in range(N):
        if inverse_arr[j] == 1:
            row[j] = 1 - row[j]
```

```
    target = 1
    #for j in range(N):
    for j in range(4):
        target = target & row[j]
    #target = 1 if np.sum(row) >= 1 else 0

    Y[i] = target

ones_idx = np.where(Y == 1)[0]
zeros_idx = np.where(Y == 0)[0]
zeros_perm = np.random.permutation(zeros_idx)
split = int(0.9 * len(zeros_idx))
train_idx = np.concatenate([ones_idx, zeros_perm[:split]])
test_idx = zeros_perm[split:]

X_train = X[train_idx]
X_test = X[test_idx]
Y_train = Y[train_idx]
Y_test = Y[test_idx]

epochs = 50000
patience = 10

initial_weights = np.random.uniform(0, 1.0, N+1)

plt.subplot()

p = Perceptron(input_size=N, X_test=X_test, Y_test=Y_test)
p.w = initial_weights.copy()
p.set_X(X_train)
p.set_target(Y_train)
p_train, p_test = p.train(epochs=epochs, patience=patience)
plt.plot(p_train, label="MSE LR Train", color="red", linestyle="--")
plt.plot(p_test, label="MSE LR Test", color="red")

adapt_p = Perceptron(input_size=N, X_test=X_test, Y_test=Y_test)
adapt_p.w = initial_weights.copy()
adapt_p.set_X(X_train)
adapt_p.set_target(Y_train)
adapt_p_train, adapt_p_test = adapt_p.train_adptive_lr(epochs=epochs,
patience=patience)
plt.plot(adapt_p_train, label="MSE ALR Train", color="black", linestyle="--")
plt.plot(adapt_p_test, label="MSE ALR Test", color="black")

X_all_bias = np.insert(X, 0, -1, axis=1)
y_pred = np.round(adapt_p.prediction(X_all_bias))
wrong = np.where(y_pred != Y)[0]
print(f"Wrong index: {wrong}")
print(f"X: {X[wrong]}, Y_true: {Y[wrong]}, Y_pred: {y_pred[wrong]}")
print(np.any(np.all(X_train == [0,1,1,1,1,1,1], axis=1)))
```

```
plt.title("Dependence MSE on epoch")
plt.xlabel("epoch")
plt.ylabel("MSE")
plt.grid(True, which="both", ls="-", alpha=0.5)
plt.legend()
plt.grid(True)

plt.show()
```

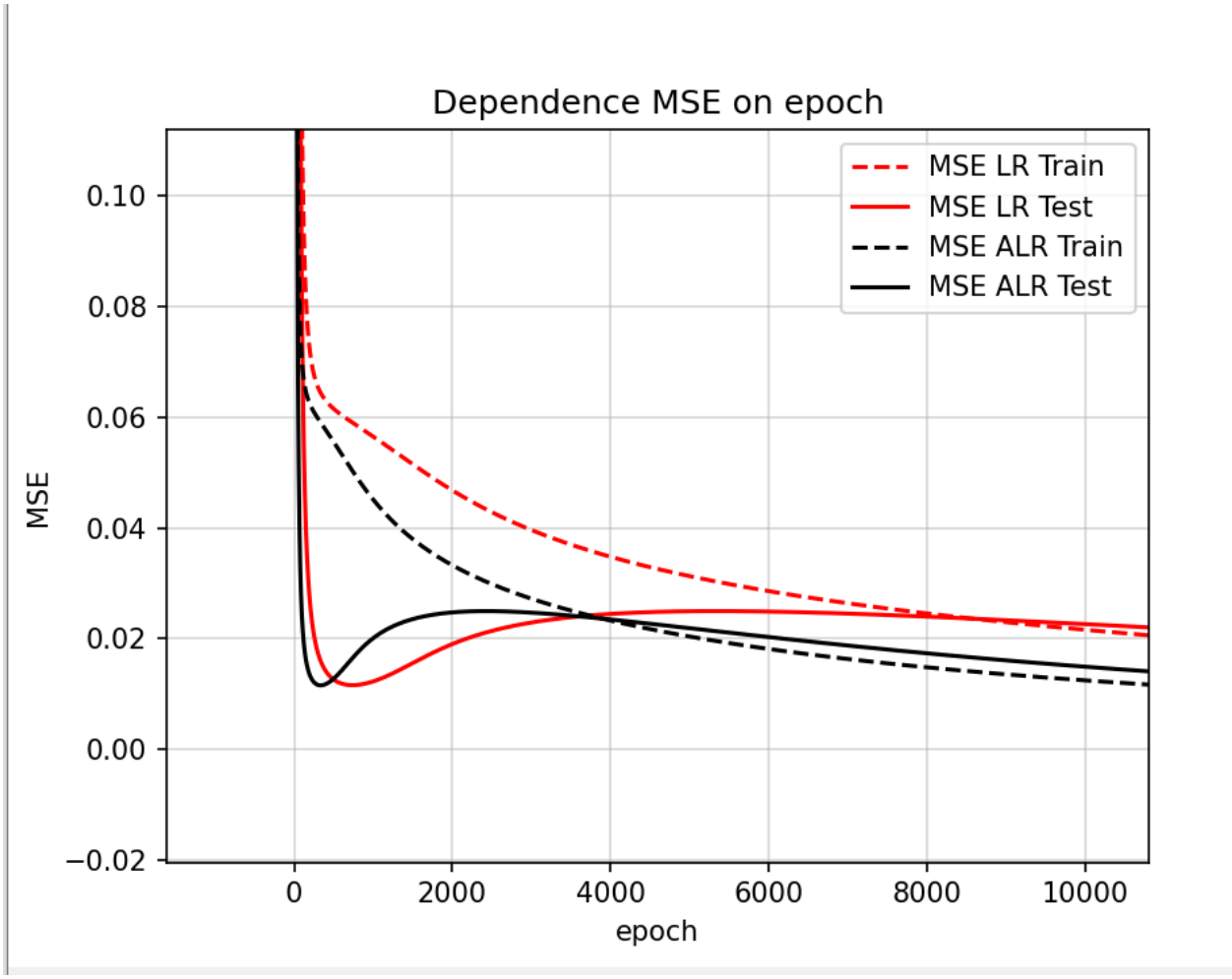
Результат:




```
[LR0.1] Stoped on 4617 epoch.
[REASON] No progress on 10 epochs.
[LR] threshold: 2.6133748316467162
[LR] weigths: [-0.53874791 -0.57988229 -0.05888039 -0.14268984 -0.22711938 -
0.51975892
 -0.35910966]
[ALR0.222007722007722] Stoped on 3112 epoch.
[REASON] No progress on 10 epochs.
[ALR] treshold: 2.726699753302443
```

```
[ALR] weigths: [-0.57161601 -0.59698175 -0.09824577 -0.17297784 -0.26520896 -
0.53884764
 -0.38799697]
Wrong index: [63]
X: [[0 1 1 1 1 1 1]], Y_true: [1.], Y_pred: [0.]
True
```

Как мы видим, из-за особенности, как обычного, так и инвертированного оператора AND, в таблице истинности вывод в основном заполнен нулями. По этой причине персептрон обучается выдавать нули. Но! Если мы увеличим содержание единиц, то получим следующий результат:



Как можно увидеть, графики тренировочных и тестовых данных стали сходиться.

Вывод: реализовал нейронную сеть (однослойный персептрон) для решения задач синтеза логических функций от n входов. Сравнить эффективность обучения при различных начальных условиях и параметрах шага.